You've been spoiled with modules in Node.js and you don't want to return to the Dark Ages. Given that you're already familiar with Node.js modules, it's worth investigating how you can get the same thing to work for your program on the client.

12.3.1. Making modules work in a browser

If web browsers don't support modules,[3] then how are you going to keep your program modular? One technique you've used in the past is to have a top-level object with a hierarchy of properties on it. Individual modules are then just properties on that top-level object.

[3] There are no modules in the fifth edition of the ECMAScript specification. See chapter 13 for a discussion of modules in upcoming editions of ECMAScript.

For example, you *might* build your client-side application by putting everything on a global `app` object, mirroring the directory structure you had on the server:

```
@app.controllers = do ->
  controller = do ->
  blog = do ->
```

In this case, your entire application can be clobbered if somebody can get source code into the application:

```
@app = loadSomeNefariousProgram()
```

That's not the biggest problem, though. The biggest problem is that if you don't enforce modularity with an actual module system, then the chances of you actually writing modular code are diminished. You need a module system that works on the client, but you'll need to implement it yourself. The module system that you've used on the server has worked well, so it makes sense to make the same thing work on the client.

How do you implement modules on the client? First, you pretend that they already work and start to write your client-side program accordingly, using the powerful design technique called wishful thinking. Once you have